

표적 궤적 모의

표적을 정의하고, 0.1초마다 움직이고, 위경도로 그리기까지

원리 · 기동 모델 · C 구현 · 결과

레이다 시스템 소프트웨어 스텀디 · Part 3

발표 대본

표지 · 약 20초

안녕하십니까. 이번 과제의 주제는 표적 궤적 모의입니다. 지난번에는 레이다가 켜 값을 위경도로 바꾸는 일을 했는데, 이번에는 반대로 표적을 우리가 직접 만들어서 60초 동안 움직여 보는 일입니다.

원리, 기동 모델, C 구현, 결과 순서로 가겠습니다.

INDEX

목차

00	들어가며	과제 3이 묻는 것
01	표적을 움직이는 원리	좌표계 넷 · 위치와 속도 시간 적분 · 지구 곡률 · 한 스텝
02	기동 모델	하중배수 · 원운동 세 축 · 기동 시간표
03	C 로 구현	구성 · 구조체 · 과제 조건 속도 변환 · 한 스텝 · 화면
04	결과	조건 · 실행 화면 · 궤적 기동 시나리오 · 검증

수학은 고등학교 물리의 등속 원운동 정도면 충분하고, 나올 때마다 다시 설명합니다.

2

발표 대본

목차 · 약 40초

다섯 부분입니다.

들어가며에서 과제가 무엇을 요구하는지 짚고, 첫째 장에서 표적을 움직이는 원리를 봅니다. 어느 좌표계를 쓰는지, 위치와 속도를 왜 다르게 다루는지, 시간을 어떻게 흘려보내는지입니다.

둘째 장은 기동 모델입니다. 표적이 방향을 바꾸는 걸 몇 g 라는 숫자 하나로 어떻게 표현하는지 다룹니다.

셋째 장은 C 구현입니다. 구조체, 과제 조건, 속도 변환, 한 스텝 함수를 코드로 보고, MFC 화면이 그걸 어떻게 돌리는지 봅니다.

넷째 장은 결과입니다. 실행 화면과 궤적 그림, 그리고 결과가 맞는지 확인한 검증표입니다.

수학은 고등학교 물리에 나오는 등속 원운동 정도면 충분합니다. 나올 때마다 다시 설명드리겠습니다.

00. 과제 3이 묻는 것

표적을 정의하고, 0.1초마다 움직이고, 위경도로 그려라



시뮬레이션 시간

60 초 · 0.1 초 간격

- $t = 0.0 \text{ s}$
- $t = 0.1 \text{ s}$
- $t = 0.2 \text{ s}$
- $t = 0.3 \text{ s}$
- $t = 0.4 \text{ s}$
- $t = 0.5 \text{ s}$
- $t = 0.6 \text{ s}$
- $t = 0.7 \text{ s}$
- $t = 0.8 \text{ s}$
- $t = 0.9 \text{ s}$
- $t = 1.0 \text{ s}$
- $t = 1.1 \text{ s}$
- $t = 1.2 \text{ s}$
- $t = 1.3 \text{ s}$
- $t = 1.4 \text{ s}$
- $t = 1.5 \text{ s}$
- $t = 1.6 \text{ s}$
- $t = 1.7 \text{ s}$
- $t = 1.8 \text{ s}$
- $t = 1.9 \text{ s}$
- $t = 2.0 \text{ s}$
- $t = 2.1 \text{ s}$
- $t = 2.2 \text{ s}$
- $t = 2.3 \text{ s}$
- $t = 2.4 \text{ s}$
- $t = 2.5 \text{ s}$
- $t = 2.6 \text{ s}$
- $t = 2.7 \text{ s}$
- $t = 2.8 \text{ s}$
- $t = 2.9 \text{ s}$
- $t = 3.0 \text{ s}$
- $t = 3.1 \text{ s}$
- $t = 3.2 \text{ s}$
- $t = 3.3 \text{ s}$
- $t = 3.4 \text{ s}$
- $t = 3.5 \text{ s}$
- $t = 3.6 \text{ s}$
- $t = 3.7 \text{ s}$
- $t = 3.8 \text{ s}$
- $t = 3.9 \text{ s}$
- $t = 4.0 \text{ s}$
- $t = 4.1 \text{ s}$
- $t = 4.2 \text{ s}$
- $t = 4.3 \text{ s}$
- $t = 4.4 \text{ s}$
- $t = 4.5 \text{ s}$
- $t = 4.6 \text{ s}$
- $t = 4.7 \text{ s}$
- $t = 4.8 \text{ s}$
- $t = 4.9 \text{ s}$
- $t = 5.0 \text{ s}$
- $t = 5.1 \text{ s}$
- $t = 5.2 \text{ s}$
- $t = 5.3 \text{ s}$
- $t = 5.4 \text{ s}$
- $t = 5.5 \text{ s}$
- $t = 5.6 \text{ s}$
- $t = 5.7 \text{ s}$
- $t = 5.8 \text{ s}$
- $t = 5.9 \text{ s}$
- $t = 6.0 \text{ s}$
- $t = 6.1 \text{ s}$
- $t = 6.2 \text{ s}$
- $t = 6.3 \text{ s}$
- $t = 6.4 \text{ s}$
- $t = 6.5 \text{ s}$
- $t = 6.6 \text{ s}$
- $t = 6.7 \text{ s}$
- $t = 6.8 \text{ s}$
- $t = 6.9 \text{ s}$
- $t = 7.0 \text{ s}$
- $t = 7.1 \text{ s}$
- $t = 7.2 \text{ s}$
- $t = 7.3 \text{ s}$
- $t = 7.4 \text{ s}$
- $t = 7.5 \text{ s}$
- $t = 7.6 \text{ s}$
- $t = 7.7 \text{ s}$
- $t = 7.8 \text{ s}$
- $t = 7.9 \text{ s}$
- $t = 8.0 \text{ s}$
- $t = 8.1 \text{ s}$
- $t = 8.2 \text{ s}$
- $t = 8.3 \text{ s}$
- $t = 8.4 \text{ s}$
- $t = 8.5 \text{ s}$
- $t = 8.6 \text{ s}$
- $t = 8.7 \text{ s}$
- $t = 8.8 \text{ s}$
- $t = 8.9 \text{ s}$
- $t = 9.0 \text{ s}$
- $t = 9.1 \text{ s}$
- $t = 9.2 \text{ s}$
- $t = 9.3 \text{ s}$
- $t = 9.4 \text{ s}$
- $t = 9.5 \text{ s}$
- $t = 9.6 \text{ s}$
- $t = 9.7 \text{ s}$
- $t = 9.8 \text{ s}$
- $t = 9.9 \text{ s}$
- $t = 10.0 \text{ s}$

위치 갱신 600 번

표적마다 · 위경도로 기록

① 표적 구조체를 정의한다

표적 최대 10개 · 표적마다 기동 최대 30개
초기 위치는 위경도·고도, 속도는 동체 속력 하나로

② 위치·속도는 ECEF 에서 갱신한다

동체 속력을 ECEF 성분으로 바꾸는 방법도 고민 (추가 항목)

③ 60초 동안 0.1초 간격으로

플랫폼 하나 (정지) · 대함 표적 · 대공 표적

④ 위경도로 출력하고 그림으로

플랫폼과 표적의 궤적을 지도 위에

과제 2 와 다른 점 : 변환 함수 넷은 제공 코드 그대로, 새로 들어온 것은 속도 · 시간 · 기동

지난번엔 측정값 하나가 체인을 한 번 지나 점 하나. 이번엔 같은 계산을 600번 반복해 선이 된다.

3

발표 대본

들어가며 · 약 70초

과제를 한 줄로 줄이면 이렇습니다. 표적을 정의하고, 0.1초마다 움직이고, 위경도로 그려라.

왼쪽 그림이 상황입니다. 우리 배는 북위 32도, 동경 126도에 서 있고 움직이지 않습니다. 북쪽으로 13.3 킬로미터 떨어진 곳에 대공 표적이 고도 300 미터에서 초속 200 미터로 남쪽, 그러니까 우리 쪽으로 날아옵니다. 그 옆에 대함 표적이 초속 30 미터로 서쪽으로 갑니다. 이것 60초 동안 0.1초 간격으로 갱신하면 표적마다 600개의 점이 찍힙니다.

오른쪽이 요구사항 네 가지입니다. 표적 구조체를 정의할 것, 표적은 최대 10개, 기동은 표적마다 최대 30번. 위치와 속도는 ECEF 좌표계에서 갱신할 것. 시뮬레이션은 60초, 0.1초 간격. 결과는 위경도로 출력하고 그림으로 보일 것.

지난 과제와 다른 점을 분명히 하고 가겠습니다. 지난번엔 측정값 하나가 변환 체인을 한 번 지나 점 하나가 나왔습니다. 이번엔 같은 계산을 600번 반복합니다. 그 변환 함수들은 제공된 참고 코드에 그대로 있어서 손댈 일이 없고, 새로 들어온 건 속도, 시간, 기동 셋입니다.

01

표적을 움직이는 원리

- 1 좌표계 넷, 그리고 왜 ECEF 인가
- 2 위치와 속도는 다르게 변환한다 — 회전 두 번
- 3 시간 적분과 지구 곡률 — 60초 직진하면 11 m 떠오른다
- 4 한 스텝의 순서 — 다섯 단계

이 발표의 약속 : 각도는 도($^{\circ}$)로 말하고 프로그램 안에서는 라디안. 거리는 m, 시간은 s.

4

발표 대본

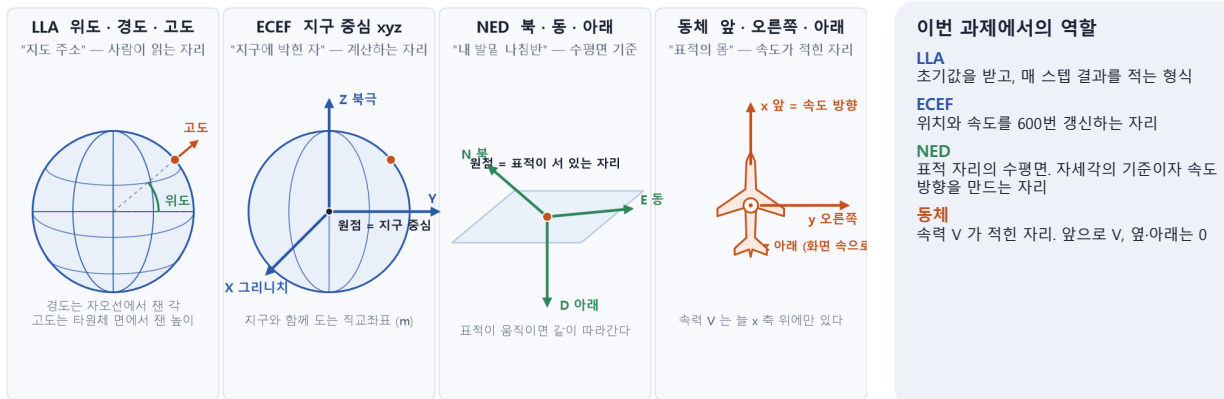
장절 · 약 25초

첫째 장입니다. 표적 하나를 0.1초 뒤로 옮기려면 뭐가 필요한지 원리부터 보겠습니다. 좌표계 넷, 위치와 속도의 차이, 시간 적분과 지구 곡률, 그리고 한 스텝의 순서까지 네 장입니다.

약속 하나만 드리면, 발표에서는 각도를 도로 말하고 프로그램 안에서는 라디안을 씁니다. 거리는 전부 미터입니다.

01. 좌표계 넷, 그리고 왜 ECEF 인가

입력/출력은 위경도, 계산은 ECEF, 수평면은 NED, 속력은 동체



왜 ECEF 인가 : 위도 $1^\circ = 111 \text{ km}$, 경도 $1^\circ = 94 \text{ km}$ (32°N) — 각도에 미터를 더할 수 없다. ECEF 는 세 축이 미터라 $p + v\Delta t$ 한 줄 플랫폼과 무관한 절대 좌표라, 나중에 배가 움직여도 표적 궤적이 흔들리지 않는다.

5

발표 대본

원리 · 약 85초

지난번에 여섯 개를 봤는데 이번에 쓰는 건 넷입니다.

첫째, 위도 경도 고도. 지도 주소라고 생각하시면 됩니다. 사람이 읽는 자리라서 과제도 처음 위치를 이 형식으로 받고 결과도 이 형식으로 내라고 합니다.

둘째, ECEF. 지구 중심에 원점을 두고 미터로 x, y, z 를 재는 직교좌표입니다. 지구에 박아 놓은 자입니다. 계산은 전부 여기서 합니다.

셋째, NED. 표적이 서 있는 자리의 북쪽, 동쪽, 아래쪽입니다. 내 발밑 나침반입니다. 원점이 플랫폼이 아니라 표적 자신이라는 게 중요합니다. 자세각이 이 수평면 기준으로 정의되니까요.

넷째, 동체. 표적의 몸 기준, 앞, 오른쪽, 아래입니다. 과제가 표적 속도를 앞으로 가는 속력 하나로 받으라고 했는데, 그게 적혀 있는 자리가 여기입니다.

그러면 왜 굳이 ECEF에서 움직이느냐. 위도는 각도입니다. 북위 32도 근처에서 위도 1도는 111 킬로미터인데 경도 1도는 94 킬로미터밖에 안 됩니다. 같은 1도라는 눈금인데 길이가 다르고, 애초에 도에다 미터를 더할 수가 없습니다. ECEF는 세 축이 전부 미터라 지금 위치에 속도 곱하기 시간을 더하면 끝입니다. 그리고 플랫폼과 무관한 절대 좌표라 나중에 배가 움직여도 표적 궤적이 흔들리지 않습니다.

01. 위치와 속도는 다르게 변환한다 — 회전 두 번

위치는 원점이 있어야 말이 되고, 속도는 화살표 하나면 된다. 그래서 속도는 회전만



속도는 화살표라서 두 번 다 회전만 한다. ③ 에서 위도·경도는 플랫폼이 아니라 표적이 지금 있는 자리의 값이다.

위치 : $p_new = R \cdot p + t$

원점이 다른 숫자가 다르다.
회전하고 원점 차이만큼 옮긴다 (과제 2).

속도 : $v_new = R \cdot v$

어디에 그러도 같은 화살표.
축이 돌아간 만큼만 돌린다. t 를 더하면 속도가 아니게 된다.

검산

결과 벡터의 크기는 항상 V (200.000).
플랫폼 위치를 잘못 더하면 6,372 km/s.

과제의 "추가 고민" (동체 속도 $V_heading \rightarrow ECEF$) 의 답 : 자세각으로 한 번, 표적 자신의 위경도로 한 번, 평행이동은 없다

참고 함수 $f_Trans_Ned_To_Ecef$ 는 플랫폼 위치를 더하므로, 속도를 넣을 때는 그 인자에 0 을 준다.

6

발표 대본

원리 · 약 90초

여기가 이번 과제의 핵심 하나입니다. 과제에 추가 고민 항목으로 나온, 동체 속도를 ECEF 성분으로 바꾸는 방법의 답이기도 합니다.

위치는 어디에 있나라는 말이라 원점이 있어야 성립합니다. 같은 점을 원점 A에서 보면 어떤 숫자고 원점 B에서 보면 다른 숫자입니다. 그래서 위치를 옮길 때는 회전행렬로 돌리고 원점 차이를 더합니다. 지난 과제에서 계속 했던 그겁니다.

속도는 어느 쪽으로 얼마나 빠리라는 말이라 원점이 필요 없습니다. 화살표 하나면 되고, 어디에 그러도 같은 화살표입니다. 그러니까 속도를 옮길 때는 축이 돌아간 만큼만 돌리면 됩니다. 회전만 하고 평행이동은 하지 않습니다.

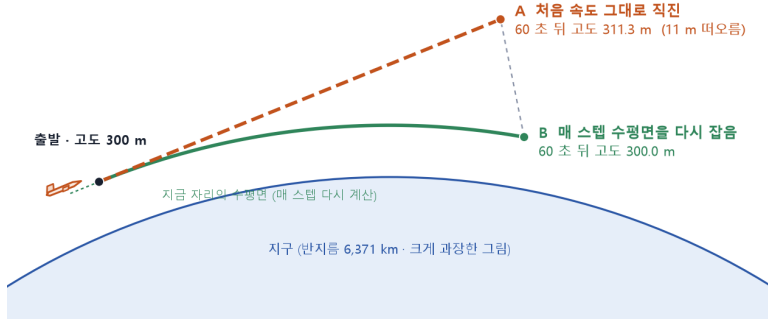
그림이 실제 순서입니다. 회전 두 번입니다. 첫째 칸, 표적은 앞으로만 가니까 동체 속도가 $V, 0, 0$ 입니다. 대공 표적이면 200, 0, 0. 둘째 칸, 자세각 롤, 피치, 요로 돌리면 NED 속도가 됩니다. 요가 180도니까 마이너스 200, 0, 0, 남쪽으로 200 이라는 뜻입니다. 셋째 칸, 표적 자신의 위도 경도로 한 번 더 돌리면 ECEF 속도입니다.

여기서 조심할 게 하나 있습니다. 참고 코드의 NED에서 ECEF로 가는 함수는 마지막에 플랫폼 위치를 더하게 되어 있습니다. 위치에는 맞는데 속도에는 그 항이 있으면 안 됩니다. 그래서 플랫폼 위치 인자에 0을 넣습니다. 검산은 간단합니다. 회전만 했으니 결과 크기가 정확히 200 이어야 합니다. 실수로 플랫폼 위치를 더하면 초속 6,372 킬로미터, 지구 반지름이 속도에 섞여 들어간 값이 나옵니다.

01. 시간 적분과 지구 곡률

새 위치 = 지금 위치 + 속도 × 0.1 초. 단, 매 스텝 수평면을 다시 잡지 않으면 60초에 11 m 떠오른다

왜 떠오르냐 : 지구가 둥글어서 접선 방향으로 12 km 가면 땅이 11 m 아래로 내려간다 ($d^2 / 2R$)
그래서 매 스텝 표적의 현재 위경도로 NED 수평면을 다시 잡고, 그 위에서 속도 방향을 다시 만든다



시간 적분 : $p(t+\Delta t) = p(t) + v(t) \cdot \Delta t$

ECEF x, y, z 에 각각. $\Delta t = 0.1$ s, 600 스텝.
한 스텝 안에서 속도가 일정하니 이 한 줄이면 정확.

실험 (대공 표적 60초 직진)

60초 뒤 고도

A 처음 ECEF 속도를 60초 내내 그대로

311.3 m

B 매 스텝 현재 위경도로 NED 를 다시 잡음

300.0 m

왜 11 m 인가

접선을 따라 d 만큼 가면 땅은 $d^2 / 2R$ 만큼 내려간다.
 $12,000^2 / (2 \times 6,371,000) = 11.3$ m. 실험값과 일치.

그래서 5단계 LLA 변환은 출력용만이 아니라 다음 스텝의 수평면(NED 축) 을 만드는 기준이다

위경도를 구해야 NED 축이 나오고, NED 축이 있어야 속도 방향이 나온다. 이 순서를 지키면 수평 비행이 수평으로 유지된다.

7

발표 대본

원리 · 약 90초

속도를 구했으니 시간을 흘려보냅니다. 새 위치는 지금 위치 더하기 속도 곱하기 0.1초. 자동차로 치면 시속 720 킬로미터로 0.1초 가면 20 미터고, 600번 더하면 12 킬로미터입니다. 한 스텝 안에서는 속도가 일정하니까 이 한 줄이면 정확합니다. 곡선은 매 스텝 방향을 다시 정해서 만듭니다.

그런데 여기 함정이 하나 있습니다. 참고 코드로 직접 돌려 본 실험입니다. 대공 표적을 고도 300 미터에서 60초 직진시켰습니다. A는 처음에 구한 ECEF 속도를 60초 내내 그대로 쓴 겁니다. B는 매 스텝 표적의 현재 위경도로 수평면을 다시 잡고 속도 방향을 다시 만든 겁니다.

A는 60초 뒤 고도가 311.3 미터입니다. 11 미터 떠올랐습니다. B는 300.0 미터입니다.

왜 떠오르느냐. 지구가 둥글기 때문입니다. 처음 자리의 수평면은 그 자리의 접선인데, 접선 방향으로 12 킬로미터를 쭉 가면 땅은 그 아래로 휘어 내려갑니다. 거리 제곱 나누기 2R, 12,000 제곱 나누기 2 곱하기 6,371,000 을 계산하면 11.3 미터입니다. 실험값과 딱 맞습니다.

그래서 매 스텝 위경도로 되돌리는 마지막 단계가 그냥 출력용이 아닙니다. 다음 스텝의 수평면을 잡는 기준입니다. 이 순서를 지키면 수평 비행이 수평으로 유지됩니다.

01. 한 스텝의 순서 — 다섯 단계

표적 하나가 0.1초 뒤로 가는 일. 이 순서를 600번, 표적 수만큼 반복한다

표적 하나의 한 스텝 (0.1 초)

왼쪽부터 순서대로. ① ②는 기동이 없으면 건너뛰고, ③ ④ ⑤는 매 스텝 반드시 한다.



① 에 필요한 것

기동 목록, 지금 시각 t

② 에 필요한 것

각속도 $\omega = n \cdot g / V, \Delta t$

③ 에 필요한 것

자세각 3개, 표적 위경도

④ 에 필요한 것

ECEF 위치-속도, $\Delta t = 0.1\text{ s}$

⑤ 에 필요한 것

ECEF → LLA 변환 함수

표적 10개 × 600 스텝 = 6,000번. 순서가 중요한 건 ③ ④ ⑤ — 자세가 정해져야 속도, 속도가 있어야 위치, 위치가 있어야 새 수평면

8

발표 대본

원리 · 약 60초

지금까지 나온 걸 한 스텝으로 정리하면 다섯 단계입니다.

하나, 기동 판단. 지금 시각이 어느 기동 구간에 들어 있는지 봅니다. 둘, 자세 갱신. 기동 중이면 각속도 곱하기 0.1초만큼 자세각을 돌립니다. 이 둘은 다음 장에서 자세히 합니다. 셋, 속도 변환. 동체에서 NED, NED에서 ECEF, 회전 두 번. 넷, 위치 적분. ECEF에서 속도 곱하기 0.1초를 더합니다. 다섯, 위경도 변환. 기록도 하고 다음 스텝의 수평면 기준도 만듭니다.

그리고 시각을 0.1초 올려 처음으로 돌아갑니다. 표적이 열 개면 6,000번인데 컴퓨터에는 아무것도 아닙니다. 순서가 중요한 건 셋, 넷, 다섯입니다. 자세가 정해져야 속도, 속도가 있어야 위치, 위치가 있어야 새 수평면입니다. 뒤에서 이 다섯 단계가 함수 하나로 그대로 나옵니다.

02

기동 모델

1 하중배수와 원운동 공식

2 세 축과 기동 시간표

약속 : 기동 중에도 속력은 바뀌지 않는다. 방향만 바뀐다.

9

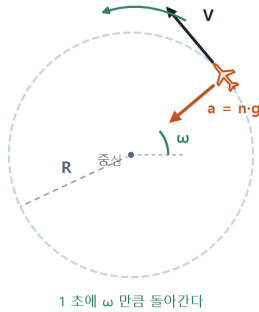
발표 대본

장절 · 약 20초

둘째 장은 기동입니다. 표적이 방향을 바꾸는 걸 어떻게 숫자로 만드느냐. 과제가 준 재료는 몇 g 인가, 어느 축인가, 언제 시작하고 끝나는가 넷입니다. 약속 하나, 기동 중에도 속력은 바뀌지 않는다고 놓습니다. 방향만 바뀝니다.

02. 하중배수와 원운동 공식

과제의 Gravity Value 는 "몇 g 로 도는가". 속력은 그대로, 방향만 바뀐다



등속 원운동 공식

구심가속도

$$a = V^2 / R = n \cdot g$$

선회 반경

$$R = V^2 / (n \cdot g)$$

각속도

$$\omega = V / R = n \cdot g / V$$

$$g = 9.80665 \text{ m/s}^2$$

V 는 변하지 않는다

표적	하중배수	각속도	선회 반경	비고
대공 200 m/s	1 g	2.81 °/s	4,079 m	180° 에 64 s
대공 200 m/s	3 g	8.43 °/s	1,360 m	180° 에 21 s
대공 200 m/s	5 g	14.05 °/s	816 m	180° 에 13 s
대함 30 m/s	0.1 g	1.87 °/s	918 m	90° 에 48 s

감 잡기 : 1 g 서 있을 때, 0.3 g 승용차 커브, 3 g 롤러코스터 바닥·전투기 급선회, 9 g 전투기 한계

가속도가 속도에 수직이라 속력은 안 변하고 방향만 변한다.

각속도가 속력에 반비례 — 배 기동에는 0.1 g 같은 작은 값을 넣어야 그럴듯하다.

프로그램에서는 한 줄 : $\omega = n \cdot g / V$ 를 구해 놓고, 매 스텝 자세각에 $\omega \times 0.1 \text{ s}$ 를 더한다 (3 g 면 한 스텝에 0.84°)

$$a = V^2 / R = n \cdot g \rightarrow R = V^2 / (n \cdot g), \omega = V / R = n \cdot g / V$$

10

발표 대본

기동 · 약 90초

과제의 Gravity Value 가 하중배수, 흔히 몇 g 라고 하는 그 숫자입니다.

커브를 도는 자동차를 생각하시면 됩니다. 속력은 그대로인데 방향이 계속 바뀌려면 안쪽으로 당기는 가속도가 있어야 합니다. 그 크기를 중력가속도 g 의 몇 배인가로 적은 게 n 입니다. 가만히 서 있으면 1 g, 승용차 커브가 0.3 g, 롤러코스터 바닥이나 전투기 급선회가 3 g, 전투기 한계가 9 g 정도입니다. 이 가속도는 속도에 수직이라 옆으로만 당기고, 그래서 속력은 안 변하고 방향만 변합니다.

그러면 고등학교 물리 등속 원운동 그대로입니다. 반경 R 을 속력 V 로 돌면 구심가속도가 V 제곱 나누기 R 이고, 이게 n g 입니다. R 에 대해 풀면 선회 반경이 V 제곱 나누기 n g, 각속도는 n g 나누기 V 입니다.

표가 실제 숫자입니다. 초속 200 미터 대공 표적이 3 g 로 돌면 1초에 8.4도, 반경 1.36 킬로미터, 반 바퀴에 21초입니다. 초속 30 미터 배는 0.1 g 만 돼도 1초에 1.9도, 반경 918 미터입니다. 같은 g 라도 느린 표적이 훨씬 빨리 돕니다. 각속도가 속력에 반비례하니까요. 그래서 배 기동에는 0.1 g 같은 작은 값을 넣어야 그럴듯합니다.

프로그램에서 하는 일은 한 줄입니다. n g 나누기 V 를 구해 놓고 매 스텝 자세각에 그 값 곱하기 0.1초를 더합니다.

02. 세 축과 기동 시간표

TurnType 은 동체의 어느 축을 돌리는가. 기동 하나 = { g, TurnType, StartTime, EndTime }, 표적마다 최대 30개

Roll (x 축 · 앞뒤 축)
뒤에서 본 그림 · 날개가 기운다

x 축 (화면 밖으로)

y 축 (화면 속으로)

속도 방향은 그대로

Pitch (y 축 · 날개 축)
동에서 본 그림 · 기수가 들린다

y 축 (화면 속으로)

z 축 (화면 속으로)

상승 · 강하

Yaw (z 축 · 위아래 축)
위에서 본 그림 · 기수가 돌아간다

yaw 각

z 축 (화면 속으로)

좌우 선회

TurnType	축	효과
0	—	직진
1 Roll	x (앞뒤)	자세만 기울
2 Yaw	z (위아래)	좌우 선회
3 Pitch	y (날개)	상승 · 강하

시간표 규칙 셋

StartTime ≤ t < EndTime 이면 적용, 밖이면 직진.

부호 : n < 0 이면 반대 방향. 겹침 : 먼저 적힌 것. 끝난 뒤 : 자세 유지.

축을 돌린 뒤 속도가 어느 쪽인지는 따로 계산하지 않는다 — 자세각만 바뀌 두면 6장의 회전 두 번이 새 속도 방향을 만든다

과제 조건에는 기동 목록이 없다 → 발표용 예시 기동을 정해 넣었다 (대공 3 g 우선회 10~20 s, 2 g 기수 들기 35~40 s, 대함 -0.1 g 좌선회 20~50 s)

11

발표 대본

기동 · 약 85초

과제의 TurnType 이 어느 축을 돌리느냐입니다. 0은 안 돌리기, 1이 롤, 2가 요, 3이 피치입니다.

피치는 날개 축을 중심으로 기수가 들리는 겁니다. 상승하거나 강하합니다. 요는 위아래 축을 중심으로 기수가 좌우로 돌아가는 겁니다. 수평 선회고, 배가 방향을 바꾸는 것도 이겁니다. 롤은 앞뒤 축, 그러니까 속도 방향 축을 중심으로 날개가 기우는 거라 속도 방향은 안 바뀌고 자세만 바뀝니다. 대신 그 뒤에 오는 피치 기동의 방향이 바뀝니다. 오른쪽으로 90도 롤을 한 다음 피치로 당기면 위가 아니라 오른쪽으로 돕니다. 실제 비행기가 선회하는 방식입니다.

축을 돌린 다음 속도가 어느 쪽인지는 따로 계산할 필요가 없습니다. 자세각만 바뀌 두면 6장에서 본 회전 두 번이 새 속도 방향을 만들어 줍니다.

기동 하나는 네 값으로 적습니다. 몇 g, 어느 축, 시작 시각, 끝 시각. 표적마다 최대 30개를 배열에 담고, 매 스텝 첫 단계에서 지금 시각이 어느 구간에 들어 있는지 봅니다. 규칙 셋을 정했습니다. g 값이 음수면 반대 방향으로 돕니다. 구간이 겹치면 먼저 적힌 걸 씁니다. 기동이 끝나면 그 순간의 자세를 유지하고 직진합니다.

과제 조건에는 기동 목록이 없습니다. 그래서 발표용 예시 기동을 우리가 정해서 넣었습니다. 안 그러면 직진 두 개만 그려지고 30개짜리 배열을 만든 의미가 안 보이니까요.

03

C 로 구현

- 1 프로그램 구성 — 제공 코드 · C 코어 · MFC 화면
- 2 표적 구조체 — target_sim.h
- 3 과제 조건 넣기 — f_SetAssignment
- 4 속도 변환 — f_BodyVelToEcef
- 5 한 스텝 — f_StepTarget
- 6 루프와 화면 — TargetSimUI

제공받은 좌표변환 코드는 한 글자도 고치지 않았다. MFC 는 계산을 하지 않는다.

12

발표 대본

장절 · 약 25초

셋째 장은 구현입니다. 코드는 C 로 짰고, 제공받은 좌표변환 코드는 한 글자도 고치지 않고 그대로 링크했습니다. 화면은 지난 과제와 같은 방식으로 MFC 로 만들었는데, MFC 쪽은 계산을 전혀 하지 않고 C 코어를 돌려서 보여 주기만 합니다.

프로그램 구성, 표적 구조체, 과제 조건, 속도 변환, 한 스텝 함수, 그리고 화면 순서로 보겠습니다.

03. 프로그램 구성

제공 코드는 무수정으로 링크, C 코어가 순서대로 부르고, MFC 는 돌려서 보여 주기만 한다

제공받은 참고 소스 코드

수정 없이 그대로 (C)

CoordinateTransform.c/.h 좌표변환 14 함수
matrixCalLib.c/.h 행렬 연산
Define.h 타입 · 상수 (G_FORCE)

링크

TargetSimCore (C 정적 라이브러리)

이번에 짰 부분 target_sim.h / .c

ST_Maneuver/Target/Scenario 구조체 셋
f_SetAssignment 과제 조건 채우기
f_StartScenario 위경도 → ECEF 초기화
f_StepScenario 0.1 초 한 스텝

호출

TargetSimUI (MFC 대화상자)

돌리고 보여 주기만 · C++

[실행] 600 스텝 루프 → 위경도 배열
지도 축 비율 1/cos(위도), 슬라이더
[재생] 100 ms 타이머, 1 ~ 10 배속
[CSV 저장] t · id · 위도 · 경도 · 고도

좌표변환은 전부 제공 코드가 하고, 코어는 그 함수를 순서대로 부르지만 한다. MFC 는 계산을 하지 않는다.

제공 코드 5 파일 = 원본 그대로

복사 후 바이트 단위 비교로 확인.
좌표변환은 이 코드만 부른다.

C 코어 250줄

구조체 셋 + 함수 다섯.
MSVC x64 · C17 · 경고 4단계 0건.

MFC 화면 (Part 2 CoordUI 와 같은 뼈대)

계산 없음. 코어를 600번 부르고 배열에 모아 그린다
지도 · 슬라이더 · 재생 · CSV 저장.

결린 것 하나 : 제공 Define.h 의 #define BOOL bool 이 MFC 의 BOOL(int) 과 충돌 → 제공 코드는 두고, MFC 쪽에서 헤더를 읽은 직후 #undef BOOL

13

발표 대본

구현 · 약 70초

세 덩어리입니다.

왼쪽이 제공받은 참고 소스 코드입니다. 좌표변환 함수와 행렬 라이브러리, 상수 정의. 다섯 파일을 수정 없이 그대로 씁니다. 실제로 원본과 바이트 단위로 같은지 비교해서 확인했습니다.

가운데가 이번에 짰 C 코어입니다. 헤더와 소스 하나씩, 합쳐서 250줄쯤 됩니다. 구조체 셋, 과제 조건을 채우는 함수, 시작 함수, 한 스텝 함수. 좌표변환은 전부 제공 코드를 부르지만 합니다.

오른쪽이 MFC 화면입니다. 실행 버튼을 누르면 코어의 스텝 함수를 600번 불러서 위경도를 배열에 모으고, 지도에 그리고, 슬라이더와 재생으로 시각을 움직입니다. CSV로 저장도 됩니다.

한 가지 짚을 게 있습니다. 제공된 Define.h 가 BOOL 을 bool 로 정의하고 있는데, 이게 MFC 의 BOOL 과 충돌합니다. 제공 코드를 고치는 대신 MFC 쪽에서 헤더를 읽은 직후 그 정의만 되돌렸습니다.

03. 표적 구조체 — target_sim.h

위 다섯 칸이 입력(사람이 적는 값), 아래 네 칸이 상태(프로그램이 갱신). 좌표 타입은 제공 코드의 구조체 그대로

표적 카드 #2

ST_Target

입력 (사람이 적는 칸)

초기 위치

32.12°N 126.0°E 300 m

동체 속도

200 m/s

초기 자세

roll 0° pitch 0° yaw 180°

기동 목록 (최대 30 개)

g	Type	Start	End
3	Yaw	10	20
2	Pitch	35	40
-			

상태 (프로그램이 갱신)

ECEF 위치

x, y, z [m]

ECEF 속도

vx, vy, vz [m/s]

현재 자세

roll, pitch, yaw

현재 위경도

출력용 · 매 스텝 변환

0.1 s

매 스텝 갱신 · 600 번

입력 칸은 건드리지 않는다

플랫폼도 속력 0 인 표적 카드

갱신 함수가 하나로 통일되고, 배가 움직이는 시나리오가 와도 카드 값만 바뀌면 된다. MAX_TARGET 10, MAX_MANEUVER 30.

```
typedef struct
{
    DOUBLE64    dGravity;        // 하중배수 [g]. 부호가 방향
    INT32        nTurnType;       // 0 없음 1 Roll 2 Yaw 3 Pitch
    DOUBLE64    dStartTime;      // [s]
    DOUBLE64    dEndTime;        // [s]
} ST_Maneuver;

typedef struct
{
    /* 입력 */
    STRUCT_Coord_Lla    stInitLla;    // 초기 위치 [rad, rad, m]
    DOUBLE64            dVheading;    // 동체 속도 [m/s]
    STRUCT_Coord_Attitude    stInitAtt;    // 초기 자세 [rad]
    INT32                nManeuverCnt;    // 기동 수
    ST_Maneuver           astManeuver[MAX_MANEUVER];    // 상태 (ECEF 에서 갱신) */
    STRUCT_Coord_Rect    stPos;        // ECEF 위치 [m]
    STRUCT_Coord_Rect    stVel;        // ECEF 속도 [m/s]
    STRUCT_Coord_Attitude    stAtt;    // 현재 자세 [rad]
    STRUCT_Coord_Lla     stLla;        // 현재 위경도 (출력용)
} ST_Target;

typedef struct
{
    ST_Target    stPlatform;        // 속력 0 인 표적 카드
    INT32        nTargetCnt;
    ST_Target    astTarget[MAX_TARGET];    // 10
    DOUBLE64    dSimTime;          // 60 s
    DOUBLE64    dDt;              // 0.1 s
} ST_Scenario;
```

14

발표 대본

구현 · 약 90초

과제가 설명을 요구한 구조체입니다. 왼쪽 그림처럼 표적 하나를 카드 한 장으로 생각하면 편합니다.

기동 하나가 ST_Maneuver 입니다. 하중배수, 축, 시작, 끝. 과제가 준 네 항목 그대로입니다.

표적 하나가 ST_Target 이고 두 칸으로 나뉩니다. 위쪽 다섯 줄이 입력 칸입니다. 초기 위치를 위경도와 고도로, 속력을 동체 앞 방향 하나로, 초기 자세를 롤 피치 요로, 그리고 기동 목록을 배열 30칸과 개수로. 과제가 요구한 입력 형식이 그대로 칸이 됩니다.

아래 네 줄이 상태 칸입니다. ECEF 위치, ECEF 속도, 현재 자세, 그리고 출력용 현재 위경도. 과제가 ECEF에서 갱신하라고 한 그 값들입니다. 처음에 입력 칸을 읽어 상태 칸을 채우고, 그 뒤 600번은 상태 칸만 고칩니다.

시나리오가 ST_Scenario 입니다. 플랫폼 카드 한 장, 표적 카드 최대 10장과 개수, 시뮬레이션 시간과 간격. 플랫폼도 속력 0인 표적 카드로 둔 게 요령입니다. 갱신 함수가 하나로 통일되고, 나중에 배가 움직이는 시나리오가 와도 카드 값만 바뀌면 됩니다.

좌표 타입은 제공 코드의 구조체를 그대로 썼습니다. 위경도, 직교좌표, 자세각 셋 다 참고 코드에 이미 있으니깐요. 이름 규칙은 코딩 규칙대로 구조체는 ST 접두사, typedef 로 선언했습니다.

03. 과제 조건 넣기 — f_SetAssignment

각도는 도로 받아 함수 안에서 라디안으로. 체크박스를 켜면 발표용 기동 예시가 붙는다

```

VOID f_SetAssignment(ST_Scenario *pstScn, INT32 bWithManeuver)
{
    memset(pstScn, 0, sizeof(*pstScn));
    pstScn->dSimTime = 60.0;
    pstScn->dDt = 0.1;
    pstScn->nTargetCnt = 2;
    /* 위도 경도 고도 속력 roll yaw pitch */
    f_SetTarget(&pstScn->stPlatform, 32.0, 126.0, 0.0, 0.0, 0.0, 0.0, 0.0);
    f_SetTarget(&pstScn->astTarget[0], 32.125, 126.03, 0.0, 30.0, 0.0, 270.0, 0.0); // 대함
    f_SetTarget(&pstScn->astTarget[1], 32.12, 126.0, 300.0, 200.0, 0.0, 180.0, 0.0); // 대공

    if (bWithManeuver) // 발표용 기동 예시
    {
        f_AddManeuver(&pstScn->astTarget[0], -0.1, TURN_YAW, 20.0, 50.0); // 대함 좌선회
        f_AddManeuver(&pstScn->astTarget[1], 3.0, TURN_YAW, 10.0, 20.0); // 대공 3 g 우선회
        f_AddManeuver(&pstScn->astTarget[1], 2.0, TURN_PITCH, 35.0, 40.0); // 대공 기수 들기
    }

    // f_SetTarget : 각도는 도로 받아 라디안으로 저장한다 (프로그램 안은 라디안)
    pstTgt->stInitIla.Lat = DEG2RAD(dLatDeg); ... pstTgt->stInitAtt.Yaw = DEG2RAD(dYawDeg);
}

```

	플랫폼	표적 1 대함	표적 2 대공
위도 · 경도	32.0 · 126.0	32.125 · 126.03	32.12 · 126.0
고도 m	0	0	300
속력 m/s	0	30	200
yaw	0	270 (서)	180 (남)

기동 예시 (화면의 체크박스)

대함 : -0.1 g yaw 20~50 s (음수 = 좌선회)
 대공 : +3 g yaw 10~20 s, +2 g pitch 35~40 s
 표적을 더 넣으려면 f_SetTarget 줄을 더 쓴다 (최대 10).

시간 60 s · 간격 0.1 s · 표적 2 개 — 과제 시뮬레이션 조건 그대로

15

발표 대본

구현 · 약 60초

과제 조건을 넣는 함수입니다. 왼쪽 코드, 오른쪽이 같은 값의 표입니다.

시뮬레이션 시간 60초, 간격 0.1초, 표적 둘. 플랫폼은 북위 32도 동경 126도, 속력 0. 표적 1이 대함, 북위 32.125도 동경 126.03도, 초속 30 미터, 요 270도로 서쪽. 표적 2가 대공, 북위 32.12도 동경 126.0도 고도 300 미터, 초속 200 미터, 요 180도로 남쪽입니다.

각도는 도로 받아서 함수 안에서 라디안으로 바꿉니다. 프로그램 안에서는 라디안만 쓴다는 약속입니다.

아래 if 문이 발표용 기동 예시입니다. 화면의 체크박스를 켜면 붙습니다. 대함 표적에 20초부터 50초까지 0.1 g 좌선회, 음수라서 왼쪽입니다. 대공 표적에 10초부터 20초까지 3 g 우선회, 35초부터 40초까지 2 g 기수 들기. 표적을 더 넣고 싶으면 이 함수에 줄을 더 쓰면 되고, 최대 10개까지입니다.

03. 속도 변환 — f_BodyVelToEcef

6장의 회전 두 번이 열 줄. 참고 코드에서 조심할 점 두 가지가 이 화면에 다 있다

① 동체 → NED

f_Trans_Body_To_Ned 에 [V, 0, 0] 과 자세각.
인자 순서는 Roll, Yaw, Pitch (헤더에 적힌 대로).

② NED → ECEF

f_Trans_Ned_To_Ecef 의 가운데 셋이 플랫폼 위치.
속도는 벡터라 여기에 0. 위경도는 표적 자신의 것.

초기화의 함정

f_Trans_Lla_To_Ecef 는 (경도, 위도, 고도) 순서.
바꿔 넣으면 북위 54° 동경 212° 가 나온다.

검산 : $|v_{ecef}| = 200.000000 \text{ m/s}$

실수로 플랫폼 위치를 더하면 6,372 km/s

```
// 동체 속력 [V, 0, 0] 을 ECEF 속도로. 회전 두 번, 평행이동 없음
static STRUCT_Coord_Rect f_BodyVelToEcef(const ST_Target *pstTgt)
{
    STRUCT_Coord_Rect stNed;

    // 1) 동체 → NED : 자세각으로 회전 (인자 순서 Roll, Yaw, Pitch)
    stNed = f_Trans_Body_To_Ned(pstTgt->dVheading, 0.0, 0.0,
                                pstTgt->stAtt.Roll, pstTgt->stAtt.Yaw, pstTgt->stAtt.Pitch);

    // 2) NED → ECEF : 표적 자신의 위경도로 회전.
    // 속도는 벡터라 플랫폼 위치 인자(가운데 셋) 에 0
    return f_Trans_Ned_To_Ecef(stNed.x, stNed.y, stNed.z, 0.0, 0.0, 0.0,
                                pstTgt->stLla.Lat, pstTgt->stLla.Lon);
}

// 초기화 : 입력 칸 → 상태 칸. f_Trans_Lla_To_Ecef 는 인자 순서가 (경도, 위도, 고도)
static VOID f_InitTarget(ST_Target *pstTgt)
{
    pstTgt->stPos = f_Trans_Lla_To_Ecef(pstTgt->stInitLla.Lon,
                                        pstTgt->stInitLla.Lat, pstTgt->stInitLla.Alt);
    pstTgt->stAtt = pstTgt->stInitAtt;
    pstTgt->stLla = pstTgt->stInitLla;
    pstTgt->stVel = f_BodyVelToEcef(pstTgt);
}
```

16

발표 대본

구현 · 약 75초

6장에서 본 회전 두 번이 이 함수입니다. 열 줄이 안 됩니다.

첫 줄이 동체에서 NED입니다. 제공 코드의 Body to Ned 함수에 속력을 x 성분으로, 나머지는 0으로 넣고, 자세각을 넘깁니다. 인자 순서가 롤, 요, 피치라서 제공 코드 헤더에 적힌 대로 맞췄습니다.

둘째 줄이 NED에서 ECEF입니다. 제공 코드의 Ned to Ecef 함수인데, 가운데 세 인자가 플랫폼 위치입니다. 속도는 벡터라 여기에 0을 넣습니다. 그 뒤가 위도 경도인데, 플랫폼이 아니라 표적 자신의 현재 위경도를 넣습니다. 표적이 서 있는 자리의 수평면이니깐요.

그 아래가 초기화입니다. 위경도를 ECEF로 바꾸는 함수는 인자 순서가 경도, 위도, 고도입니다. 구조체는 위도가 먼저라 바꿔 넣기 쉬운데, 바꾸면 북위 54도 동경 212도라는 엉뚱한 점이 나옵니다. 참고 코드에서 조심할 점 두 가지가 이 화면에 다 있습니다.

03. 한 스텝 — f_StepTarget

8장의 다섯 단계가 함수 하나. 주석 번호가 8장 번호와 같다

① ② 기동 판단 · 자세 갱신

for 문. 구간 안이면 $\omega \cdot dt$ 만큼 그 축을 돌리고 break (먼저 적힌 기동 우선). $+1e-6$ 은 0.1을 600번 더한 부동소수점 찌꺼기 여유.

③ 속도 변환

매 스텝 다시 부른다. 자세가 안 바뀌어도 표적의 위경도가 바뀌니까.

④ 위치 적분

ECEF 세 성분 $v \times dt$. 이 세 줄이 시간 적분.

⑤ LLA 변환

기록되고, 다음 스텝 ③의 수평면 기준이 된다 (지구 곡률 처리).

속력 0인 플랫폼도 같은 함수를 지낸다 — 기동 판단에 속력 0이면 건너뛰는 조건이 있고, 속도가 0이라 자리만 지킨다.

```
static VOID f_StepTarget(ST_Target *pstTgt, DOUBLE64 dTime, DOUBLE64 dDt)
{
    INT32 i;

    // 1) 기동 판단, 2) 자세 갱신 : 각속도  $\omega = n \cdot g / V$ 
    for (i = 0; i < pstTgt->nManeuverCnt; i++)
    {
        const ST_Maneuver *pstMan = &pstTgt->astManeuver[i];

        if ((dTime + 1e-6 >= pstMan->dStartTime) && (dTime + 1e-6 < pstMan->dEndTime)
            && (pstTgt->dVheading > 0.0))
        {
            DOUBLE64 dAng = pstMan->dGravity * G_FORCE / pstTgt->dVheading * dDt;

            if (pstMan->nTurnType == TURN_ROLL)    pstTgt->stAtt.Roll += dAng;
            else if (pstMan->nTurnType == TURN_YAW) pstTgt->stAtt.Yaw += dAng;
            else if (pstMan->nTurnType == TURN_PITCH) pstTgt->stAtt.Pitch += dAng;
            break;                                // 먼저 적힌 기동 하나만
        }
    }

    // 3) 속도 변환 (표적 자신의 현재 위경도 기준 NED)
    pstTgt->stVel = f_BodyVelToEcef(pstTgt);

    // 4) 위치 적분 (ECEF)
    pstTgt->stPos.x += pstTgt->stVel.x * dDt;
    pstTgt->stPos.y += pstTgt->stVel.y * dDt;
    pstTgt->stPos.z += pstTgt->stVel.z * dDt;

    // 5) LLA 변환. 출력용이자 다음 스텝의 NED 기준
    pstTgt->stLla = f_Trans_Ecef_To_Lla(pstTgt->stPos.x, pstTgt->stPos.y, pstTgt->stPos.z);
}
```

17

발표 대본

구현 · 약 90초

8장의 다섯 단계가 함수 하나입니다. 주석의 번호가 8장 번호와 같습니다.

첫째와 둘째가 for 문입니다. 기동 목록을 처음부터 훑다가 시작 시각 이하, 끝 시각 미만인 기동을 만나면, 각속도 $n \cdot g$ 나누기 V 에 0.1초를 곱한 만큼 그 축의 자세각을 더하고 break로 빠져나옵니다. 먼저 적힌 기동이 우선이라는 규칙이 이 break입니다. 시각 비교에 붙인 $1e-6$ 은 0.1을 600번 더하면서 생기는 부동소수점 찌꺼기 때문입니다.

셋째, 방금 본 속도 변환 함수를 부릅니다. 자세가 바뀌었든 안 바뀌었든 매 스텝 부릅니다. 표적의 현재 위경도가 매 스텝 달라지니까요.

넷째, ECEF 세 성분에 속도 곱하기 dt 를 더합니다. 위치 적분이 이 세 줄입니다.

다섯째, 제공 코드로 ECEF를 위경도로 되돌립니다. 이 값이 기록되고, 다음 스텝 셋째 단계의 수평면 기준이 됩니다. 7장의 지구 곡률 문제를 해결하는 게 바로 이 순서입니다.

속력이 0인 플랫폼도 같은 함수를 지냅니다. 기동 판단에 속력 0이면 건너뛰는 조건이 있고, 속도가 0이니 위치가 안 변할 뿐입니다.

03. 루프와 화면 — TargetSimUI (MFC)

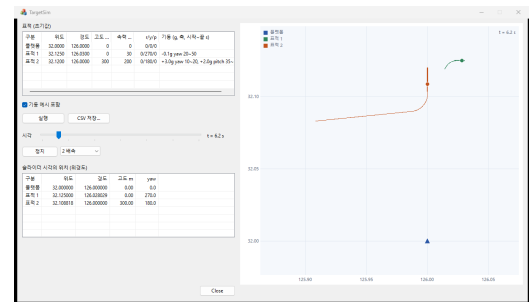
MFC 는 계산이 없다. 코어를 600번 부르고, 배열에 모으고, 지도에 그리고, 시각을 움직인다

```
// TargetSimUIDlg.cpp [실행] : 60 초를 0.1 초 간격으로 돌리며 매 스텝 위경도를 모은다
const int nSteps = (int)floor(m_stScn.dSimTime / m_stScn.dDt + 0.5); // 600

f_StartScenario(&m_stScn); // 위경도 → ECEF, 속도 초기화
record(); // t = 0
for (int k = 0; k < nSteps; k++)
{
    f_StepScenario(&m_stScn, k * m_stScn.dDt); // 플랫폼 + 표적 전부 한 스텝
    record(); // 위도 · 경도 · 고도 · yaw 를 배열에
}
m_map.SetPaths(&m_paths); // 지도에 레직, 슬라이더 0 ~ 600
```

```
// MapView.cpp 위도 1 도가 경도 1 도보다 1/cos(위도) 배 길다 → 축 비율
const double dAsp = 1.0 / cos(dLatC * PI / 180.0);
if (dLatSpan * dAsp * dW / dH > dLonSpan) dLonSpan = dLatSpan * dAsp * dW / dH;
else dLatSpan = dLonSpan / dAsp * dH / dW;

// 재생 : 100 ms 타이머 한 틱에 배속(1.2~5.10) 만큼 스텝 → 1 배속 = 실시간
SetTimer(TIMER_PLAY, 100, nullptr);
```



2 배속 재생 중 (t = 6.2 s). 점의 짧은 선이 진행 방향.

CSV 저장

t, id, 위도, 경도, 고도, yaw 를 한 줄씩 — 과제가 요구한 위경도 출력.
뒤의 파이썬 그림은 이 파일로 그렸다.

지도의 축 비율 $1/\cos(\text{위도})$ 를 안 맞추면 선회 원이 타원으로 보인다. 재생은 1 배속이 실시간, 10 배속까지.

18

발표 대본

구현 · 약 70초

MFC 쪽은 계산이 없습니다. 실행 버튼의 루프가 위 코드입니다. 시작 함수로 상태 칸을 채우고, 600번 스텝 함수를 부르면서 매번 위경도와 고도, 요 각을 배열에 담습니다. 그게 전부입니다. 배열이 채워지면 지도에 넘기고 슬라이더 범위를 0부터 600으로 잡습니다.

지도에서 한 가지 신경 쓴 게 축 비율입니다. 5장에서 본 대로 위도 1도가 경도 1도보다 길기 때문에, 위도 축을 1 나누기 코사인 위도 배 늘려 그림니다. 안 그러면 선회 원이 찌그러진 타원으로 보입니다.

재생은 100 밀리초 타이머입니다. 한 틱에 배속만큼 스텝을 넘기는데, 1배속이면 시뮬레이션 0.1초에 실제 0.1초라 실시간이고 10배속까지 있습니다. 오른쪽이 2배속으로 재생 중인 화면입니다. 표적 위치의 짧은 선이 진행 방향입니다.

CSV 저장은 시각, 표적 번호, 위도, 경도, 고도, 요 각을 한 줄씩 적습니다. 과제가 요구한 위경도 출력이 이 파일이고, 뒤에 나올 파이썬 그림은 이 파일로 그렸습니다.

04

결과

1 시뮬레이션 조건

2 실행 화면

3 궤적과 고도

4 기동 시나리오

5 검증

화면의 값 = CSV 의 값 = 파이썬 그림의 값. 같은 코드가 만든 값은 같다.

19

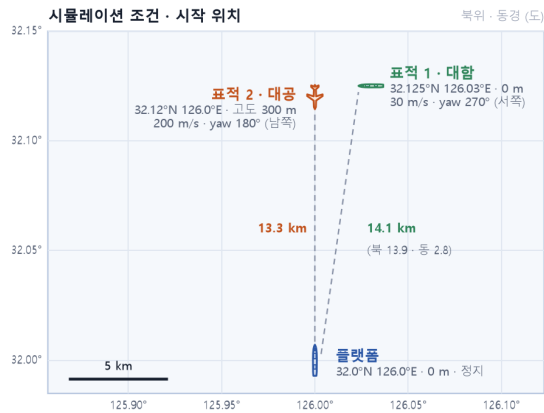
발표 대본

장절 · 약 20초

넷째 장은 결과입니다. 과제 조건을 지도에 놓고, 실행 화면을 보고, 궤적과 고도 그림을 보고, 기동을 넣은 시나리오를 보고, 마지막으로 결과가 맞는지 확인한 검증표입니다.

04. 시뮬레이션 조건 — 플랫폼 하나, 표적 둘

대공 표적은 정면으로 접근하고, 대함 표적은 정북 방향을 가로지른다



항목	플랫폼	대함 표적 (1)	대공 표적 (2)
위도 · 경도	32.0°N 126.0°E	32.125°N 126.03°E	32.12°N 126.0°E
고도	0 m	0 m	300 m
속력 (동체 앞)	0 m/s	30 m/s	200 m/s
자세각 (r · y · p)	—	0° · 270° · 0° (서쪽)	0° · 180° · 0° (남쪽)
시간	60 s, 간격 0.1 s (600 스텝)		

플랫폼 기준으로 바꿔 보면

대공 : 정북 13.3 km, 고각 1.2° → 정확히 우리 쪽으로 온다 (정면 접근)

대함 : 북 13.9 km · 동 2.8 km, 거리 14.1 km → 서쪽으로 가며 정북을 가로지른다

두 표적의 출발점은 550 m · 2.8 km 차이로 아주 가깝다

같은 자리에서 출발해 하나는 남쪽으로 빠르게, 하나는 서쪽으로 천천히 — 그림에서 대비가 분명하도록 짜인 조건

20

발표 대본

결과 · 약 60초

과제가 준 조건을 지도 위에 놓은 겁니다.

플랫폼이 북위 32도 동경 126도, 정지. 대공 표적이 정북 13.3 킬로미터, 고도 300 미터, 남쪽으로 초속 200 미터. 정확히 우리 쪽으로 날아오는 정면 접근입니다. 처음 고각이 1.2도입니다. 대함 표적은 북쪽 13.9 킬로미터, 동쪽 2.8 킬로미터, 거리 14.1 킬로미터에서 서쪽으로 초속 30 미터. 우리 정북 방향을 가로지릅니다.

두 표적 출발점이 아주 가깝습니다. 550 미터, 2.8 킬로미터 차이입니다. 같은 자리에서 하나는 남쪽으로 빠르게, 하나는 서쪽으로 천천히 가니까 그림에서 대비가 분명합니다.

04. 실행 화면 — 과제 조건, $t = 60\text{ s}$

왼쪽 위 표적 카드의 입력 칸, 오른쪽 궤적 지도, 왼쪽 아래 슬라이더 시각의 상태 칸



상태 칸의 값이 그대로 [CSV 저장] 으로 나간다 — 과제 요구 "시뮬레이션 결과를 LLA 로 출력"

21

발표 대본

결과 · 약 70초

과제 조건으로 실행한 화면입니다. 왼쪽 위가 표적 카드의 입력 칸, 오른쪽이 궤적 지도, 왼쪽 아래가 슬라이더 시각의 상태 칸입니다. 슬라이더가 60초 끝에 가 있습니다.

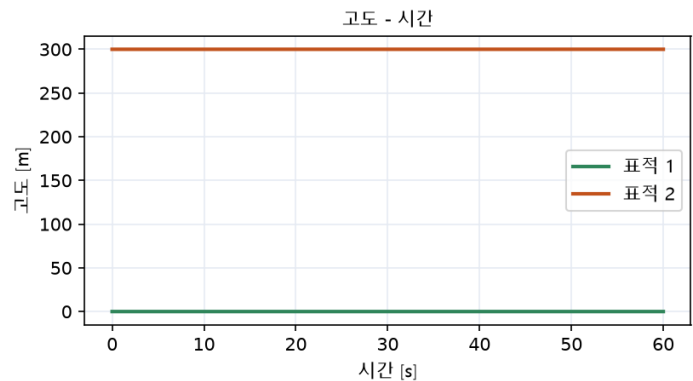
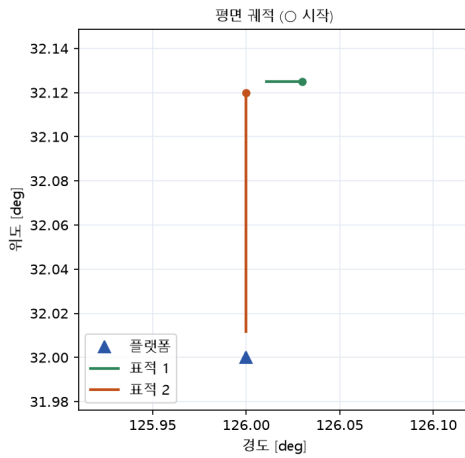
지도를 보시면 대공 표적의 주황 선이 정남으로 내려와 플랫폼 삼각형 바로 위에서 끝납니다. 대함 표적의 초록 선은 오른쪽 위에서 서쪽으로 짧게 갑니다. 선 길이 차이가 속력 차이 그대로입니다.

아래 표의 숫자를 읽어 보겠습니다. 대공 표적은 60초 뒤 북위 32.011788도, 경도는 그대로 126도입니다. 플랫폼에서 1,341 미터 앞이고 고각은 12.9도까지 올라갔습니다. 거의 머리 위입니다. 고도는 300.02 미터입니다. 대함 표적은 위도 그대로, 경도가 126.010925도, 서쪽으로 1.8 킬로미터 갔습니다.

이 표의 값이 그대로 CSV로 저장됩니다.

04. 궤적과 고도 — CSV 를 파이썬으로

평면 궤적은 축 비율을 맞춰서, 고도는 평평해야 한다 (지구 곡률 처리의 증거)



60 s 뒤 고도 300.019 m — 2 cm 의 정체

한 스텝에 20 m 접선으로 가서 $20^2 / 2R$ 씩 뜨고, 600번이면 1.9 cm. 이론값과 같다.

매 스텝 수평면을 다시 잡지 않았다면 이 선이 300 → 311 m 로 기울었을 것

22

발표 대본

결과 · 약 60초

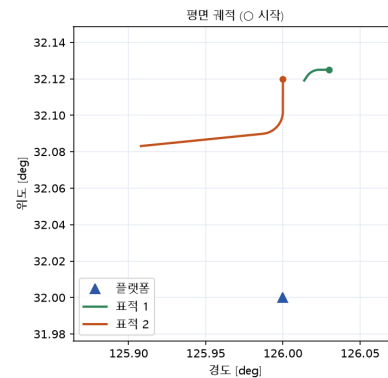
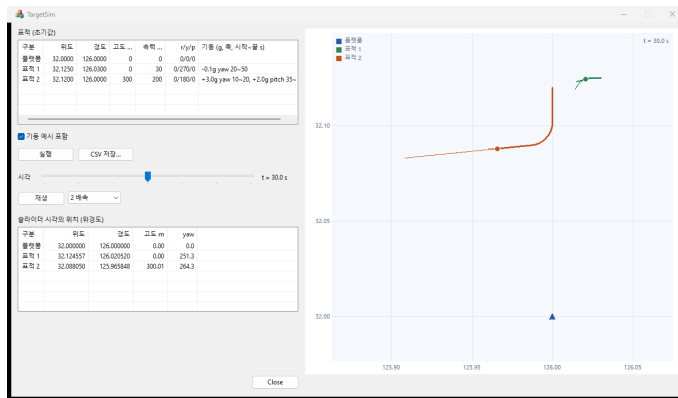
저장한 CSV를 파이썬으로 그린 겁니다. 왼쪽이 평면 궤적, 오른쪽이 고도와 시간입니다.

평면 궤적은 화면 지도와 같은 모양인데 축 비율을 맞춰 봤습니다. 위도 축이 경도 축보다 1.18배 길게 그려져 있어서 실제 거리 비율이 맞습니다.

고도 그림이 중요합니다. 대공 300 미터, 대함 0 미터에서 완전히 평평합니다. 7장에서 매 스텝 수평면을 다시 잡지 않으면 11 미터 떠오른다고 했는데, 그게 처리됐다는 증거가 이 평평한 선입니다. 숫자로는 60초 뒤 300.019 미터, 2 센티미터 차이입니다. 이 2 센티미터도 이유가 있습니다. 한 스텝에 20 미터를 접선 방향으로 가니까 스텝마다 20 제곱 나누기 2R 만큼 뜨고, 그게 600번이면 1.9 센티미터입니다. 이론값과 같습니다.

04. 기동 시나리오 — 3 g 우선회, 기수 들기, 좌선회

체크박스를 켜고 실행. 왼쪽은 $t = 30$ s 화면, 오른쪽은 CSV 로 그린 궤적



대공 3 g 우선회 (10~20 s)

10초에 84.3° 돌아 서쪽으로.
궤적 세 점 외접원 1,359.5 m vs $V^2/(n-g)$ 1,359.6 m

대공 2 g 기수 들기 (35~40 s)

5초에 28.1° 올라간 뒤 그 각으로 20초.
60 s 에 고도 2,428.7 m

대함 -0.1 g 좌선회 (20~50 s)

30초에 -56.2° 남서쪽으로.
반경 917.8 m vs 이론 917.7 m

23

발표 대본

결과 · 약 80초

체크박스를 켜고 기동 예시를 넣은 결과입니다. 왼쪽이 화면, 30초 시점입니다. 오른쪽이 파이썬 그림입니다.

대공 표적을 보시면 10초에 3 g 우선회가 시작돼서 10초 동안 84도를 돌아 서쪽을 향하고, 그 뒤로 직진합니다. 이 원호의 반지름을 궤적 세 점의 외접원으로 재 보니 1,359.5 미터고, 이론값 V 제곱 나누기 $n-g$ 는 1,359.6 미터입니다. 0.01 퍼센트 차이입니다. 10장의 공식이 코드에서 그대로 재현된 겁니다.

35초부터 5초 동안 2 g 로 기수를 들었습니다. 5초에 28도가 올라가고, 그 뒤 20초를 28도로 올라가니까 60초에 고도가 2,428 미터가 됩니다. 고도 그림에서 35초부터 올라가는 선이 그겁니다.

대함 표적은 20초부터 50초까지 0.1 g 좌선회입니다. 30초 동안 56도를 돌아 남서쪽을 봅니다. 반경 918 미터, 이론값과 같습니다.

화면 왼쪽 위의 기동 목록 옆에 이 값들이 그대로 보입니다. 발표 중에 슬라이더를 움직여 표적이 도는 걸 직접 보여 드리겠습니다.

04. 검증 — 다섯 가지로 확인

좌표변환은 틀려도 그럴듯한 값이 나온다 (과제 2의 교훈). 저장한 CSV 로 계산했다

항목	어떻게 확인하나	기대값	결과
① 이동 거리	ECEF 위치 차이의 합 vs 속도 × 시간	$200 \times 60 = 12,000 \text{ m}$	12,000.000 m (차 1e-9 m)
② 속도 보존	매 스텝 $ v_{\text{ecef}} $ vs 입력 속도	200 m/s	최소-최대 200.000000 (편차 1e-14)
③ 고도 유지	수평 비행 60 s 뒤 고도	300 m	300.019 m (이론 +1.9 cm)
④ 선회 반경	기동 시작-중간-끝 세 점의 외접원 vs $V^2/(n \cdot g)$	3 g : 1,359.6 m	1,359.5 m (-0.01 %)
⑤ 위경도 왕복	LLA → ECEF → LLA → ECEF 거리	1 mm 미만	3e-5 m

왜 미리 정하나

코드가 내놓을 값을 먼저 알고 있어야 코드가 맞는지 알 수 있다.
①②③⑤는 개념편에서 손계산한 값과 같고, ④는 기동 코드가 틀린 뒤 잦다.

한 번 틀리면 바로 드러나는 실수들

경도·위도 인자 순서 바꿈 → 북위 54° 동경 212°. 속도에 플랫폼 위치 더함 → 6,372 km/s.
수평면을 한 번만 잡음 → 고도 311 m.

다섯 칸이 채워졌으니 궤적이 "그림" 이 아니라 "결과" 다

24

발표 대본

결과 · 약 70초

지난 과제에서 얻은 교훈이 하나 있습니다. 좌표변환은 틀려도 그럴듯한 값이 나온다. 이번엔 점이 600개라 눈으로는 더 못 잡습니다. 그래서 확인 항목을 다섯 개 정해서 저장한 CSV로 계산했습니다.

첫째, 이동 거리. 속도 곱하기 시간이니까 대공 표적은 12,000 미터를 가야 합니다. 경로 길이가 12,000.000 미터, 오차 1 나노미터 수준입니다. 대함은 1,800.000 미터.

둘째, 속도 보존. 매 스텝 ECEF 속도의 크기가 입력 속력과 같아야 합니다. 최소 최대 모두 200.000000, 편차가 1e-14 입니다. 회전만 했다는 증거입니다.

셋째, 고도 유지. 300.000 에서 300.019 미터, 앞에서 설명한 2 센티미터입니다.

넷째, 선회 반경. 1,359.5 대 1,359.6 미터.

다섯째, 위경도 왕복. 위경도를 ECEF로 갔다가 되돌리면 같은 값이어야 하는데 오차가 0.03 밀리미터입니다. 참고 코드는 반복을 다섯 번 고정으로 하는데 이 정도면 충분합니다.

다섯 칸이 채워졌으니 궤적이 그림이 아니라 결과입니다.

정리

- ① 과제 3 = 과제 2의 변환 함수 + 시간 루프 + 기동 모델. 제공 코드는 한 줄도 안 고쳤다
- ② 위치는 회전 + 평행이동, 속도는 회전만. 속도에 위치를 더하면 안 된다
- ③ 새 위치 = 지금 위치 + 속도 $\times$ 0.1 초. ECEF 에서 더한다
- ④ 지구는 둥글다. 매 스텝 표적 자리의 수평면을 다시 잡아야 고도가 유지된다
- ⑤ 기동은 원운동. $\omega = n_g / V$, $R = V^2 / (n \cdot g)$. 코드가 0.01 % 로 재현했다
- ⑥ 표적 카드 = 입력 칸 + 상태 칸. 플랫폼도 카드 한 장
- ⑦ 검증 없는 궤적은 그림일 뿐. 거리 · 속력 · 고도 · 반경 · 왕복 다섯 가지로 확인했다

질문 환영합니다

코드 · 문서 : [radar-sw-study / Part3_TargetSim](#)

25

발표 대본

정리 · 약 60초

일곱 가지로 정리하겠습니다.

하나, 과제 3은 과제 2의 변환 함수에 시간 루프와 기동 모델을 얹은 것입니다. 제공 코드는 한 줄도 안 고쳤습니다. 둘, 위치는 회전과 평행이동, 속도는 회전만입니다. 속도에 위치를 더하면 안 됩니다. 셋, 새 위치는 지금 위치 더하기 속도 곱하기 0.1초, ECEF에서 더합니다. 넷, 지구는 둥글습니다. 매 스텝 표적 자리의 수평면을 다시 잡아야 고도가 유지됩니다. 다섯, 기동은 원운동입니다. 각속도 n_g 나누기 V , 반경 V 제곱 나누기 n_g . 코드가 이 값을 0.01 퍼센트로 재현했습니다. 여섯, 표적 카드는 입력 칸과 상태 칸으로 나눕니다. 플랫폼도 카드 한 장입니다. 일곱, 검증 없는 궤적은 그림일 뿐입니다. 거리, 속력, 고도, 반경, 왕복 다섯 가지로 확인했습니다.

이상입니다. 질문 환영합니다.

부록 A. 참고 코드의 함수와 함정

제공된 CoordinateTransform.c 에서 이번에 쓴 것과, 컴파일해서 확인한 주의점

함수	이번 과제의 용도	주의
f_Trans_Lla_To_Ecef (Lon, Lat, Alt)	초기 위치 입력	경도가 첫째 인자. 바꾸면 북위 54° 동경 212°
f_Trans_Ecef_To_Lla (X, Y, Z)	매 스텝 출력 + 다음 수평면 기준	5회 고정 반복. 우리 위치에서 왕복 오차 3e-5 m
f_Trans_Body_To_Ned (... , Roll, Yaw, Pitch)	동체 속도 → NED 속도	인자 순서가 Roll, Yaw, Pitch
f_Trans_Ned_To_Ecef (... , Pf_X, Pf_Y, Pf_Z, Lat, Lon)	NED 속도 → ECEF 속도	속도에는 Pf_X·Y·Z = 0. 아니면 6,372 km/s
Ant 계열 4개 · Enu 계열 2개 · PfCompensation · Ecef_To_Ned	사용 안 함	표적을 만드는 쪽이라 안테나 좌표계가 없다

각도는 전부 라디안 (DEG2RAD 매크로는 target_sim.h 에 정의) · Body → NED 행렬은 Rz(yaw)·Ry(pitch)·Rx(roll) 과 소수점 6자리까지 일치 (과제 2 와 같은 규약)

그 밖에 : 한글 주석이 UTF-8 이라 MSVC /utf-8 옵션 (vcxproj 에 포함). Define.h 의 #define BOOL bool 은 MFC 와 충돌 → MFC 쪽에서 #undef. G_FORCE = 9.80665 는 Define.h 에 있다.

26

발표 대본

부록 · 약 40초

제공된 좌표변환 코드에서 이번에 쓴 함수와 조심한 점입니다.

쓰는 건 넷입니다. 위경도와 ECEF를 오가는 둘, 동체에서 NED, NED에서 ECEF. 안테나 계열과 ENU 계열, 플랫폼 보상 함수는 안 썼습니다.

함정이 넷입니다. 위경도에서 ECEF로 가는 함수는 경도가 첫째 인자입니다. 자세각 인자는 롤, 요, 피치 순서입니다. 모든 각도는 라디안입니다. NED에서 ECEF로 가는 함수는 플랫폼 위치를 더하므로 속도에는 0을 넣어야 합니다. 그리고 Define.h 의 BOOL 정의가 MFC와 충돌해서 MFC 쪽에서 되돌렸습니다.

부록 B. 용어 정리

발표에 나온 말들. 질문이 들어오면 이 표를 보면서 답한다

용어	뜻	이번 과제에서
하중배수 n (g)	안쪽으로 당기는 가속도가 중력가속도의 몇 배인가	기동의 Gravity Value. $a = n \cdot g$
각속도 ω	1초에 몇 도(라디안) 도는가	$\omega = n \cdot g / V$. 매 스텝 자세각 $\pm \omega \times 0.1 \text{ s}$
오일러 적분	새 값 = 지금 값 + 변화율 $\times$ 시간 간격	$p \leftarrow p + v \times \Delta t$. 스텝 안 속도 일정이라 정확
자세각 roll · pitch · yaw	동체 축이 수평면(NED)에 대해 돌아간 세 각	표적의 진행 방향. yaw 180° = 남쪽
ECEF	지구 중심 원점, 지구와 함께 도는 직교좌표 [m]	위치·속도를 갱신하는 자리
NED	한 지점의 북 · 동 · 아래 수평 좌표	표적 자신의 자리 기준. 매 스텝 다시 잡는다
타원체고	WGS-84 타원체 면에서 수직으로 잰 높이	과제의 "고도". 해발고도와 약 20 m 차이
회전행렬 R	한 좌표계의 축을 다른 좌표계 축으로 돌려 읽는 3×3	위치엔 $R \cdot p + t$, 속도엔 $R \cdot v$

이상입니다. 질문 환영합니다. 코드 · 문서 : radar-sw-study / Part3_TargetSim

27

발표 대본

부록 · 약 30초

오늘 나온 용어를 한 표로 모았습니다. 하중배수, 각속도, 오일러 적분, 자세각, ECEF, NED, 타원체고, 회전행렬. 질문이 있으면 이 표를 보면서 답하겠습니다.

발표에서는 말하지 않는다. 질문이 들어올 때만 쓴다.

Q1. 왜 룬지쿠타 4차를 안 쓰고 오일러 한 줄로 하나?

룬지쿠타는 가속도가 위치의 함수일 때, 그러니까 미분방정식을 풀어야 할 때 필요하다. 위성 궤도가 그 경우다. 이번 과제는 속도를 매 스텝 우리가 직접 정해 주므로 스텝 안에서 속도가 상수이고, 상수 속도의 적분은 곱셈 한 번으로 정확하다. 남은 오차는 선회 구간에서 방향이 스텝 안에서 조금씩 변하는 것뿐인데, 0.1초에 0.84도(3 g, 200 m/s)라 원호를 600개 직선으로 잇는 오차는 반경 대비 0.01 % 수준이고 실제 검증에서도 0.01 % 가 나왔다.

Q2. 롤 기동을 어떻게 해석했나?

롤은 속도 방향 축(동체 x) 둘레의 회전이라 궤적을 직접 바꾸지 않고 자세만 바꾼다. 대신 그 뒤에 오는 피치 기동의 방향을 바꾼다. 롤 90도 뒤 피치는 수평 선회가 된다. 실제 항공기 선회 방식(기울이고 당기기)과 같다. 롤 각속도도 다른 축과 같이 $n \cdot g / V$ 로 두었는데 이는 정의의 문제라 다른 값으로 바뀌도 구조는 같다.

Q3. g 값의 부호, 기동이 겹칠 때, 끝난 뒤는?

부호는 방향이다. 양수면 오른쪽(요), 기수 들기(피치), 오른쪽 날개 내리기(롤), 음수면 반대. 겹치면 배열에서 먼저 적힌 기동 하나만 적용한다 (for 문의 break). 끝나면 그 순간 자세를 유지하고 직진한다. 원래 방향으로 복귀시키려면 반대 부호 기동을 하나 더 적으면 된다. 셋 다 과제가 정해 주지 않아서 우리가 정한 규칙이고 코드 주석과 발표에 명시했다.

Q4. 11 m 떠오른다는 값의 근거는?

접선을 따라 거리 d 만큼 가면 지표면은 $d^2/(2R)$ 만큼 아래로 내려간다. $d = 12,000 \text{ m}$, $R = 6,371,000 \text{ m}$ 이면 11.3 m. 참고 코드로 실제로 60초 적분한 값이 $311.33 - 300 = 11.33 \text{ m}$ 로 일치한다. 매 스텝 위경도로 NED 를 다시 잡으면 300.019 m 다. 남은 1.9 cm 는 스텝마다 접선 방향으로 20 m 를 가서 생기는 $20^2/(2R)$

x 600 이고 이론값과 같다.

Q5. f_Trans_Ned_To_Ecef 에 플랫폼 위치를 0으로 넣는 이유는?